

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

04 Types and coercion

Modules 1-2: type systems and operator semantics**1. Introduction & core concept**

A type classifies values and the operations that can be meaningfully applied to them. A type system specifies rules for constructing and checking expressions. Static checking rejects some violations before execution; dynamic checking detects others while operations run. These dimensions do not imply that statically typed languages reject every bad program or that dynamically typed languages lack rules.

Coercion is a conversion applied implicitly by a language rule; explicit conversion is requested in the program. Language designers balance convenience, predictability, compatibility, and error detection. The same operator spelling may select addition, concatenation, or a user-defined operation, depending on operand types and conversion rules.

2. Under the hood: the mechanics

Parsing establishes expression structure; type analysis or runtime dispatch determines how an operation applies. A Python name can later be rebound to an object of another type, while each actual operation still uses operand-specific behavior. JavaScript specifies abstract conversion operations such as `ToNumber` and `ToString` [2]. C++ uses static types, overload resolution, promotions, and conversion rules.

Representation also matters. Mathematical integers, fixed-width signed/unsigned values, floating-point numbers, and strings obey different laws and limits. Distinguish a rejected operation, a successful but surprising conversion, lost precision, and an operation outside the language's defined guarantees. Calling all of them 'type errors' obscures the actual mechanism.

Learning outcomes

- Distinguish static/dynamic checking from implicit/explicit conversion.
- Predict operator results from operand types.
- Explain why a compiling C++ comparison can still violate intent.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: rebinding does not imply coercion

```
value = 5
print(value + 2)

value = "5"
try:
    print(value + 2)
except TypeError:
    print("converted:", int(value) + 2)
```

In-depth reasoning

Expected result: 7
converted: 7

Initially `value` refers to an integer, so `value + 2` performs integer addition. The assignment of a string afterward changes the binding, not the type of the previous integer object. When the second addition is evaluated, the operands are now a string and an integer.

Python does not implicitly turn this integer into a string for str concatenation, nor parse the string as an integer for this addition. Evaluation raises `TypeError` before `print` receives a result. The handler explicitly calls `int` on the string, obtains the integer 5, adds 2, and prints the recovered result.

Explicit conversion has its own domain: replacing the string with `'five'` causes `int` to raise `ValueError`. That new exception occurs inside the handler and is not caught by the preceding `TypeError` handler. Type compatibility and textual validity are separate conditions. The original string remains a string after `int` returns a new numeric value.

Compare the language models

JavaScript's plus operator can concatenate after implicit conversion for these operands, producing `'52'`.

C++ generally rejects `std::string + int` because no matching standard string operation is available. Similar source text therefore has different semantics.

Checkpoint: Would `str(2)` fix the intended arithmetic? No; it requests concatenation-compatible text, yielding `'52'`.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: conversion depends on the operator

```
console.log("5" + 2);  
console.log("5" - 2);  
console.log("0" == false);  
console.log("0" === false);  
console.log(Number(""));  
console.log(Number.isNaN(Number("five")));
```

In-depth reasoning

Expected result: 52

3

true

false

0

true

The plus expression has a string operand, so it produces concatenated text after the applicable conversion. Subtraction instead requests numeric operands; '5' becomes 5 and the result is 3. An operator's spelling is therefore insufficient to determine a result without considering its conversion rules.

Loose equality permits conversions: for '0' and false, the comparison eventually operates on matching numeric zero values. Strict equality does not perform that type-coercing comparison, so a string and a Boolean are unequal. Strict equality still has special numeric rules; it should not be described as identical to bitwise or object-identity comparison in every case.

Number("") yields zero, which can be surprising in input processing. Number('five') yields NaN, a numeric invalid-result value, and Number.isNaN detects it. A conversion returning a value is not evidence that the application's required input format was satisfied. These are specified outcomes, not random engine behavior.

Compare the language models

Python int("") rejects empty text, whereas this JavaScript conversion accepts it as zero. C++ conversion APIs differ: strict parsing with from_chars can check both an error code and complete consumption. Validate the chosen grammar before relying on conversion.

Checkpoint: Why prefer an explicit conversion followed by validation at an input boundary? It makes both the intended type and acceptance policy visible.

3. CODE EXAMPLES / C++

Advanced: mixed signedness changes a comparison

```
#include <iostream>

int main() {
    int debt = -1;
    unsigned int limit = 1;
    std::cout << std::boolalpha;
    std::cout << (debt < limit) << '\n';

    bool below = debt < 0 ||
        static_cast<unsigned int>(debt) < limit;
    std::cout << below << '\n';

    double rate = 3.9;
    int whole = static_cast<int>(rate);
    std::cout << whole << '\n';
}
```

In-depth reasoning

Expected result: false

true

3

For this int/unsigned int pair, the usual arithmetic conversions convert the signed operand to unsigned before comparison. Converting -1 produces the maximum unsigned int value, so the first comparison is false [3]. The program can compile successfully while the result conflicts with the intended mathematical comparison.

The second expression handles negative signed values first. Short-circuit OR means the cast is evaluated only when debt is nonnegative. Every nonnegative int value fits the corresponding unsigned int, so that branch compares without the negative-value surprise. This is a domain-aware comparison policy, not a general invitation to silence warnings using casts.

The explicit floating-to-integer conversion truncates 3.9 toward zero, producing 3. For this representable value the operation is defined. Arbitrary floating values need range and validity checks before conversion. List-initializing an int from the variable rate would instead be rejected as a narrowing conversion. Explicit conversion and lossless conversion are different claims.

Compare the language models

Python built-in integers do not have this fixed-width signed/unsigned pairing. JavaScript ordinary Number comparisons also do not follow C++'s unsigned promotion rules. Predicting an expression requires the language's conversion model, not an informal label such as 'strongly typed'.

Checkpoint: What if debt is 0? The first OR operand is false, and the checked nonnegative branch compares 0 with 1.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

Use three questions: what types do the operands have, which conversion rules apply, and can the chosen representation express the intended result? The Types and coercion demonstration intentionally contrasts string-plus-integer behavior. Its errors are teaching evidence, not proof that one language is universally safer.

Review questions with answer guidance

1. Are dynamic typing and implicit coercion the same?

No. Dynamic checking concerns when types govern operations; coercion concerns whether conversions happen implicitly.

2. Does a cast prove that a value is in range?

No. A cast expresses an operation; the program must establish its preconditions.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: built-in numeric and sequence types. [Read source](#)

[2] ECMAScript: type conversion operations. [Read source](#)

[3] C++ working draft: usual arithmetic conversions. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.